Gonzalo Argañaraz

Senior Backend Engineer (.NET/C#) | Entrega con IA nativa y flujos de trabajo multiagente

Córdoba, Argentina (UTC-3) | Abierto a remoto, full-time (relación de dependencia o contractor) | Inglés C1
arggondev@gmail.com | linkedin.com/in/arggon | github.com/Arggon | arggon.github.io/arggon-cv

RESUMEN

Ingeniero backend senior .NET/C# que entrega features de punta a punta con flujos de trabajo multiagente de IA (Claude Code, GitHub Copilot, OpenCode). Creador de ArggonManager, una metodología spec-driven para agentes de código con IA, y diseñador de un pipeline de SDLC multiagente en Coalfire (pruebas de concepto). Fuerte en APIs, performance de SQL Server (bajé el tiempo de procesamiento del cliente un 80% con un endpoint por lotes) y en verificar la salida de la IA con tests y review. Remoto desde Córdoba, Argentina (UTC-3), disponible para solapar con el horario laboral de EE.UU.

EXPERIENCIA

Software Engineer to Senior AI Software Engineer | GlobalLogic | Apr 2022 – Aug 2026
Remoto. Empresa de servicios de ingeniería; asignado a proyectos de clientes de EE.UU.

Coalfire (ciberseguridad y cumplimiento) | Senior AI Software Engineer | May 2026

- Me encargué del diseño de los agentes y de la metodología de entrega en un equipo de IA de tres personas, junto a un project manager que presentó la propuesta al cliente y un ingeniero que probó los agentes contra la metodología.
- Diseñé un pipeline de SDLC multiagente: un agente de transcripción extrae specs de las transcripciones de reuniones, un agente de backlog las convierte en work items, un agente developer implementa y abre un pull request, un agente de QA lo valida con el Playwright CLI y un agente tech-lead revisa el PR. Cualquier etapa devuelve el trabajo si falla un chequeo.
- Armé el pipeline sobre una metodología Spec-Driven Development (SDD), redactada con Claude Sonnet y validada con pruebas de concepto en GitHub Copilot (GPT-4o).
- Extendí la metodología de SDLC multiagente con test-driven development: un comportamiento a la vez, primero el test que falla, después la implementación y después el refactor.
- Refactoricé las skills de los agentes para priorizar el relevamiento de requisitos y un alcance de feature claro antes de generar código.

Coalfire | Senior Software Engineer | Jul 2025 – Dec 2025

- Migré endpoints críticos de una API monolítica legacy a un microservicio .NET 9, bajando los tiempos de respuesta un 30% bajo carga alta.
- Diseñé e implementé un rol de acceso nuevo, con sus permisos y restricciones, que habilitó la monetización a través de las ofertas default de AWS Marketplace.

Lytx (telemática de video y seguridad de flotas) | Software Engineer II | Apr 2022 – May 2025

- Diseñé y construí un endpoint de datos masivo que reemplazó las llamadas una por una de los clientes por un solo request por lote, respaldado por un stored procedure nuevo en SQL Server que procesa el conjunto completo en la base, bajando el tiempo de procesamiento del cliente un 80%.
- Corrí operaciones async independientes en paralelo con Task.WhenAll e integré servicios backend con funciones AWS Lambda existentes.
- Mentoré a un desarrollador mid-level con pair programming en features de negocio y problemas de backend en C#.
- Armé filtrado avanzado para que los usuarios encontraran rápido los datos que necesitaban, en lugar de recorrer resultados grandes.
- Diagnostiqué y corregí issues que atravesaban frontend, backend y base de datos, mejorando la estabilidad del sistema.

Software Developer | e.tres (e-commerce) | 2021 – 2022

- Rediseñé la lógica central de transacciones para corregir problemas de integración con la pasarela de pagos, mejorando la confiabilidad y bajando las fallas de integración.
- Refactoricé partes críticas del codebase para alinearlas con los requisitos de negocio, mejorando la legibilidad y la mantenibilidad.

PROYECTOS

dotnet-batchyard | Proyecto de portfolio, open source | .NET 10, SQL Server, YARP, k6, docker compose, GitHub Actions | github.com/Arggon/dotnet-batchyard

- Construí un showcase open source en .NET 10 (docker compose, GitHub Actions) que saca la ingesta por lotes de un monolito legacy con el patrón strangler detrás de un gateway YARP; en pruebas k6 sobre una VM en la nube, con 500 ítems por request, la latencia mediana bajó 93% (unos 300 ms a 20 ms) y el throughput subió de unos 7.7k a 95k ítems/s. Hecho con mi metodología ArggonManager.
- Armé setups de Docker y pipelines de CI/CD con ayuda de agentes de IA en trabajo de prueba de concepto (este repo es el ejemplo público).

ArggonManager | Foco actual | TypeScript, open source | github.com/Arggon/ArggonManager

- Una metodología para entregar software con agentes de IA y humanos en git, soportada por un CLI, un servidor MCP y plugins de agentes. Corre en OpenCode, ZCode y Claude Code a través de un kernel de reglas compartido, así las reglas se mantienen iguales entre herramientas.
- Los roles se definen por lo que deciden: delivery lead, practice and standards, agentes maker y verifier, más un product owner humano. La documentación se dimensiona por riesgo (solo task, spec y plan, o exploración con un gate de análisis de specs), y las reviews quedan registradas en el work item como veredictos con evidencia: comandos ejecutados, esperado versus observado.
- Done gate: un ítem no puede cerrarse mientras quede algún criterio de aceptación sin tildar. Eximir ese gate, tomar un claim ajeno, reabrir trabajo terminado y hacer releases son solo para humanos, y lo hace cumplir la herramienta, no el wording del prompt. Construido y trackeado con ella misma: 27 ADRs, 25 specs, publicado en npm con release-please.

HABILIDADES

- **Backend:** C#, .NET 9 / .NET Core, ASP.NET Core Web API, Entity Framework, diseño de REST API, microservicios, CQRS, patrón Mediator, Clean Architecture, SOLID
- **Datos y mensajería:** SQL Server (T-SQL, stored procedures), Redis, RabbitMQ, Kafka
- **Cloud y DevOps:** AWS (API Gateway, Lambda, S3, AWS Marketplace), Docker, CI/CD, Git
- **Ingeniería asistida por IA:** flujos de SDLC agentic, orquestación multiagente, metodología ArggonManager, spec-driven development, TDD para código generado por IA, MCP, agent skills, OpenCode, ZCode, Claude Code, GitHub Copilot, Claude Sonnet, GPT-4o
- **Testing y observabilidad:** xUnit, NUnit, Playwright, Vitest, Jasmine, Datadog, New Relic
- **Seguridad y auth:** OAuth, JWT, IdentityServer, control de acceso basado en roles
- **Frontend:** Angular 19, TypeScript, JavaScript, Angular Material, PrimeNG, HTML, CSS
- **También:** Rust (aprendiendo), Go, Python (proyectos personales)

EDUCACIÓN

Técnico Programador Universitario | Universidad Tecnológica Nacional, Facultad Regional Tucumán (UTN FRT), Argentina | 2021

IDIOMAS Y CURSOS

- Español (nativo), Inglés C1 (certificado EF SET)
- Cursos (Udemy): Getting Started with .NET Core Clean Architecture; Enterprise Application Architecture with .NET Core